TP — Exploiter des données alimentaires inspirées d'INCA/Ciqual

Durée indicative : 3 h — Niveau : Python / structures de données / bases de données

Travail : binôme

1. Contexte

On souhaite exploiter deux types d'informations complémentaires :

- des informations sur des individus et leurs consommations ;

- des informations sur la composition nutritionnelle des aliments.

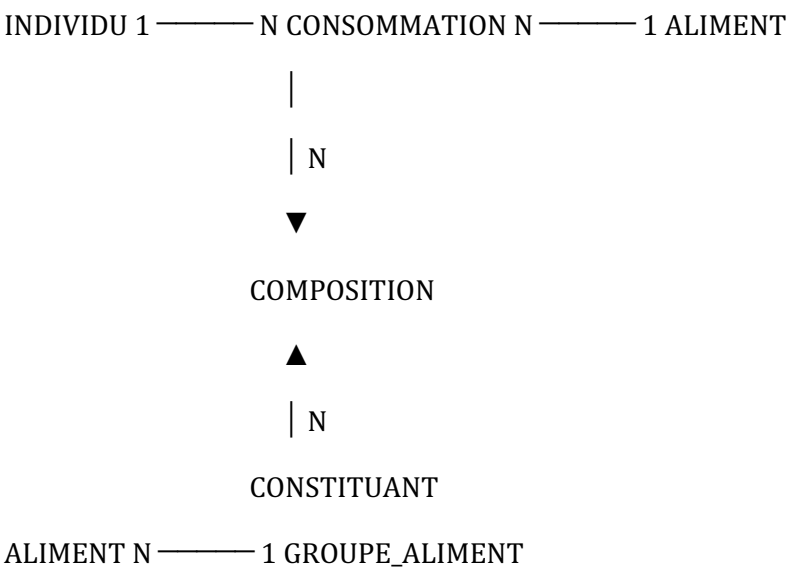
Le jeu fourni dans ce TP est volontairement synthétique. Il reproduit la logique générale d'un rapprochement entre données de consommation et données de composition.

> Important : les données du TP ne sont pas les données officielles INCA/Ciqual et ne doivent pas être interprétées comme telles.

---

2. Le modèle de données

Les fichiers représentent les relations suivantes :



Clés

- `individus.id_individu` : clé primaire

- `aliments.code_aliment` : clé primaire

- `groupes.code_groupe` : clé primaire

- `constituants.code_const` : clé primaire

- `consommations.id_conso` : clé primaire

- `composition.(code_aliment, code_const)` : clé primaire composée

Partie A — Découverte des données

A1. Chargement

Charger les six fichiers CSV dans des DataFrame.

Afficher :

- leur nombre de lignes ;

- leur nombre de colonnes ;

- leurs noms de colonnes.

A2. Inspection

Pour chaque table, utiliser `head()` et `info()`.

Questions :

1. Quelle table contient les individus ?

2. Quelle table décrit les aliments ?

3. Quelle table donne les valeurs nutritionnelles ?

4. Pourquoi `composition` possède-t-elle deux codes ?

Partie B — Recherche

B1. Rechercher un aliment

Compléter :

def rechercher_aliment(texte):

    ...

La fonction doit retourner les aliments dont le libellé contient le texte recherché, sans tenir compte des majuscules/minuscules.

Exemples :

rechercher_aliment("pain")

rechercher_aliment("POMME")

B2. Composition d'un aliment

Compléter :

def composition_aliment(code_aliment):

    ...

Le résultat doit contenir :

- `code_const`

- `nom`

- `teneur`

- `unite`

Tester avec le pain (`1001`).

Partie C — Calcul nutritionnel

Les teneurs sont exprimées pour 100 g.

Pour une quantité `q` :

apport = teneur × q / 100

C1. Apport d'un aliment

Compléter :

def apport_aliment(code_aliment, quantite_g):

    ...

Pour 60 g de pain, vérifier notamment que l'apport en protéines est :

8,5 × 60 / 100 = 5,1 g

C2. Apports d'un individu

Compléter :

def apports_individu(id_individu):

    ...

La fonction doit :

1. sélectionner les consommations de l'individu ;

2. retrouver les aliments ;

3. retrouver leur composition ;

4. calculer l'apport correspondant à chaque consommation ;

5. additionner les apports pour chaque constituant.

Partie D — Les jointures

Réécrire `apports_individu()` en utilisant explicitement `merge()`.

On cherche à obtenir conceptuellement :

consommations

    │

    ├── merge aliments

    │

    └── merge composition

            │

└── merge constituants

Questions :

1. Quelle colonne permet la jointure avec `aliments` ?

2. Quelle colonne permet la jointure avec `composition` ?

3. Pourquoi `composition` augmente-t-elle fortement le nombre de lignes ?

4. Que représente une ligne du DataFrame final ?

Partie E — Analyse par repas

Compléter :

```
def apports_par_repas(id_individu):
    ...
```

La fonction doit produire les apports par repas.

Exemple de structure attendue :

repas	Energie	Proteines	Lipides
petit_dejeuner	...		
collation	...		
dejeuner	...		
diner	...		

Partie F — Questions d'analyse

À partir des fonctions développées, répondre par Python aux questions suivantes.

1. Combien d'aliments différents sont consommés par l'individu 1 ?

2. Quel est son apport total en énergie ?

3. Quel est son apport total en protéines ?

4. Quel repas lui apporte le plus d'énergie ?

5. Quel aliment contribue le plus à son apport protéique ?

6. Quel individu consomme la plus grande quantité totale d'aliments ?

7. Quel individu possède l'apport énergétique total le plus élevé ?

Pour les questions 6 et 7, expliquer l'algorithme utilisé.

Partie G — Qualité des données

Le fichier `consommations.csv` constitue un jeu simplifié.

Imaginez maintenant que certaines données réelles puissent contenir :

- un aliment sans code correspondant dans la table des aliments ;

- une quantité nulle ;

- une quantité négative ;

- un code constituant absent ;

- une teneur manquante ;

- plusieurs lignes identiques.

Écrire une fonction :

```
def controler_donnees():
    ...
```

qui détecte au moins trois catégories d'anomalies.

Partie H — Défi algorithmique

Écrire une fonction :

```
def aliment_plus_proteine(id_individu):
    ...
```

Elle doit retourner l'aliment ayant fourni la plus grande quantité de protéines à l'individu.

Contrainte : utiliser au moins une jointure et une agrégation.

Partie I — Extension facultative

Sans utiliser `groupby`, écrire une fonction qui calcule la quantité totale de protéines consommée par un individu.

Puis comparer cette solution à celle utilisant `groupby`.

Discussion :

- laquelle est la plus lisible ?

- laquelle nécessite le plus de code ?

- quel rôle joue `groupby` ?

- quelle structure algorithmique utiliseriez-vous sans pandas ?

Livrable

Déposer :

1. `inca_ciqual.py`

2. éventuellement un notebook `TP_INCA_CIQUAL.ipynb`

3. une courte réponse aux questions d'analyse

4. les tests réalisés

Le programme doit être exécutable sans modification des fichiers de données.

Critères de qualité

On valorisera :

- des fonctions courtes et correctement nommées ;

- l'absence de code inutilement répété ;

- des noms de variables explicites ;

- des résultats vérifiables ;

- la gestion des cas particuliers ;

- des commentaires lorsque le raisonnement n'est pas évident.